Autonomous Networking

Networks that observe, decide, and adapt

Gaia Maselli
Department of Computer Science
Sapienza University of Rome

A.Y. 2026 – 2027

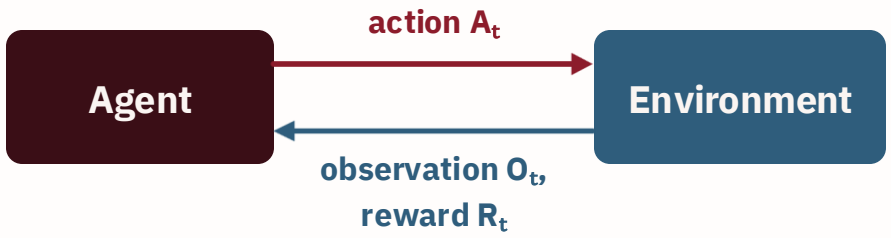
SAPIENZA
UNIVERSITÀ DI ROMA



RECAP

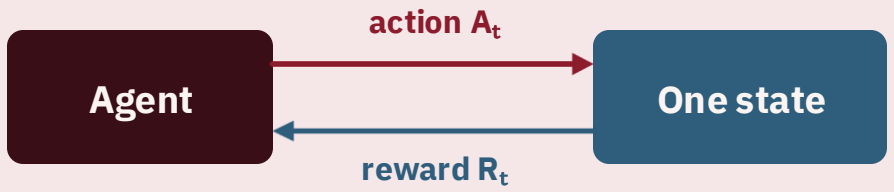
From full RL to the bandit problem

LAST LECTURE · FULL RL



History H_t, agent state $S_t = f(H_t)$, policy π, return G_t: what you do now changes what you face next

TODAY · BANDIT



No state, no delayed reward: only **which action pays more**, learned from rewards. The exploration–exploitation dilemma in its simplest form

TODAY'S QUESTION

How can we balance exploration with exploitation?

Exploit

Use what you know: pick the action that looks best now

Explore

Try something else: it might turn out to be better

TODAY'S GOALS

By the end of this lecture you can…

1. **Formulate** a decision problem as a k-armed bandit: actions, rewards, action values $q_*(a)$
2. **Estimate** action values from rewards, incrementally, with constant memory
3. **Apply** greedy and ε-greedy selection, and compute how often each action is chosen
4. **Choose** the step size α when the rewards change over time
5. **Explain** how ε-greedy and optimistic initial values each answer today's question

TODAY'S PLAN

A bandit algorithm has two ingredients

Estimate

How good is each action, given what I have seen?

- Sample average
- Incremental update
- Nonstationary: constant step size α

estimates $Q_t(a)$

new reward R_t

Select

Which action do I take now?

- Random · Greedy
- ε-greedy
- Optimistic initial values

Exploration vs. exploitation is decided by the **selection** rule, but it can only be as good as the **estimates** it relies on.

EXPLORATION VS. EXPLOITATION

Why the agent must explore

Evaluative feedback

Rewards **evaluate** the action taken: how good it was, not which action was best

Depends on the action

You only learn about the actions you actually try

Active exploration

The agent must search explicitly for good behavior

Should the agent explore or exploit? To study it, we use a **simplified version** of the RL problem.

THE BANDIT PROBLEM

The k-armed bandit problem

1

Choose

You face, repeatedly, a choice among **k** different options, or actions

2

Get a reward

A number drawn from a **stationary** probability distribution that depends on the action

Goal

Maximize

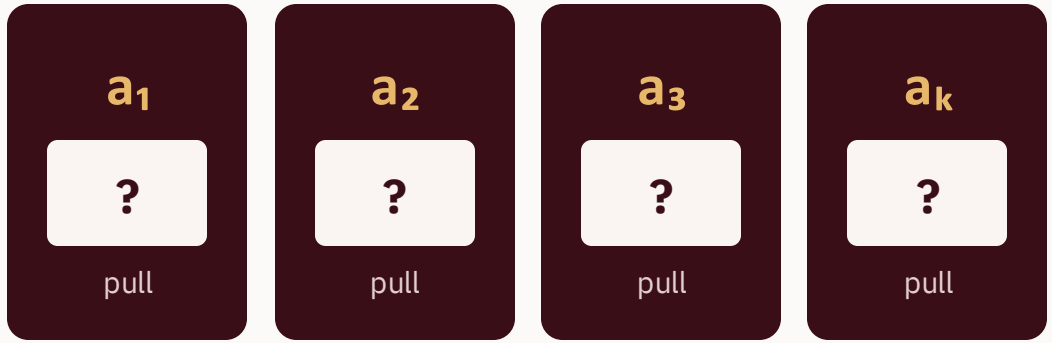
the expected total reward over a time period, e.g. 1000 action selections (time steps)

⚠ **Not just finding the best arm:** every step counts. Rewards are collected **while** learning, so exploration has a cost.

Stationary: the reward distributions do not change over time. We relax this later.

WHERE THE NAME COMES FROM

A slot machine with k levers



A "one-armed bandit" is a slot machine. Here it has **k levers** instead of one.

Each pull is an action; the payout is the reward. Each lever has its own, **unknown**, payout distribution.

By repeated pulls, concentrate your actions on the **best levers**.

FORMALIZATION

The ingredients and the notation

THE PROBLEM

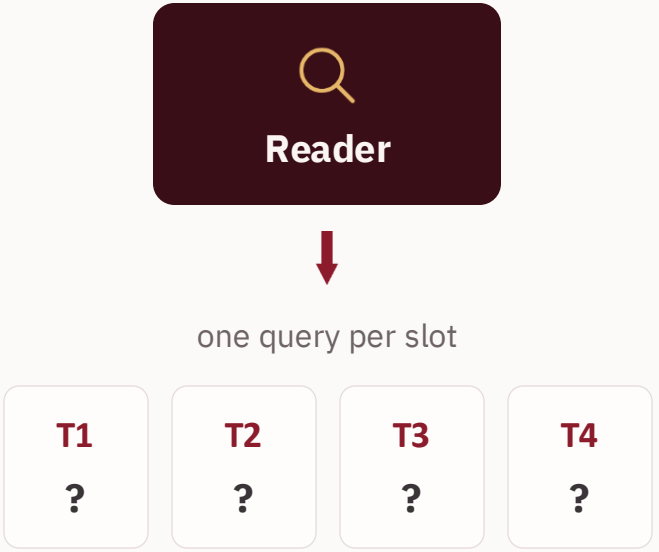
- A set of **k actions** $A = \{a_1, \ldots, a_k\}$
- Each action has an **unknown** reward distribution
- A **single state**: the situation never changes
- At each step t the agent picks A_t and gets R_t
- Goal: maximize the **expected total reward**

NOTATION

A_t, R_t	action and reward at step t
$q^*(a)$	true value of a (unknown)
$Q_t(a)$	our estimate of $q^*(a)$ at step t
$N_t(a)$	times a was chosen before t
ε	exploration probability
α	step size (learning rate)

BACK TO LECTURE 1

The smart-home reader is a bandit



Agent	the reader
Arms (actions)	the tags: query tag i in this slot
Reward	+1 new data, −1 nothing new
q*(i)	how productive tag i is, on average
Exploit	query the tags that were rich in data
Explore	query a tag we know little about

Simplification for today: each tag has new data with a **fixed** probability. We revisit this at the end. For the numbers we use a smaller example: **a doctor**.

RUNNING EXAMPLE

A doctor choosing among treatments

Agent

the doctor

ACTIONS · K = 3 TREATMENTS

- **Red**
- **Yellow**
- **Blue**

REWARD

The patient's **welfare** after the treatment. Easier to measure: the **change in blood pressure**.

Each new patient is a new step t: same situation, same three choices, a noisy outcome.

ACTION VALUES

How good is each action?

To decide which treatment is best, the doctor needs a **value** for each action: the **expected** (mean) reward obtained by taking it.

$$q^*(a) \doteq \mathbb{E}[\, R_t \mid A_t = a \,]$$

the value of action a is the expected reward, given that a is selected

If we knew q*

The problem would be trivial: always pick $\text{argmax}_a\, q^*(a)$

vs. last lecture

$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$: here no state and no future, so the return is just one reward

But we don't

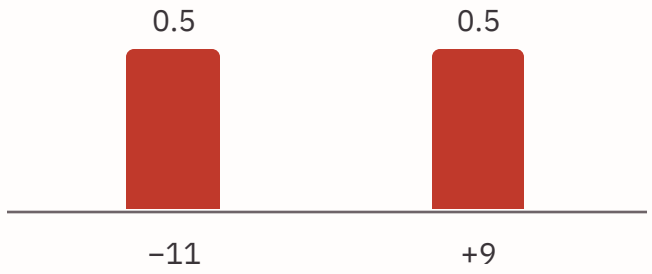
We must **estimate** it from the rewards we observe: $Q_t(a) \approx q^*(a)$

ACTION VALUES

q*(a) is the mean of each distribution

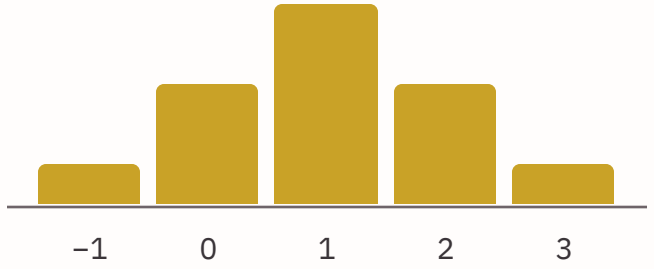
Each treatment yields rewards (change in blood pressure) from a different distribution:

Red · Bernoulli



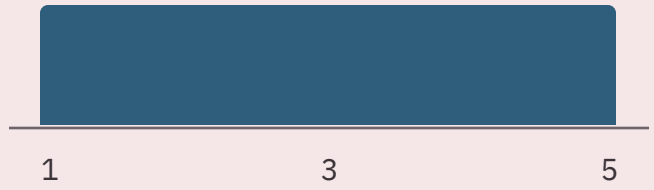
0.5·(−11) + 0.5·9 = **−1**

Yellow · Binomial



symmetric around 1 → **1**

Blue · Uniform



(1 + 5) / 2 = **3** → best

ESTIMATING ACTION VALUES

The sample-average estimate

The distributions are unknown, so the doctor runs many trials and averages the rewards actually observed for each treatment.

$$Q_t(a) \doteq \frac{\text{sum of rewards when a taken before t}}{\text{number of times a taken before t}} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{1}[A_i=a]}{\sum_{i=1}^{t-1} \mathbb{1}[A_i=a]}$$

$\mathbb{1}$[predicate]

1 if the predicate is true, 0 otherwise: it keeps only the steps where a was chosen

Denominator = 0

a never tried yet: define $Q_t(a) = 0$ (or another default)

Law of large numbers

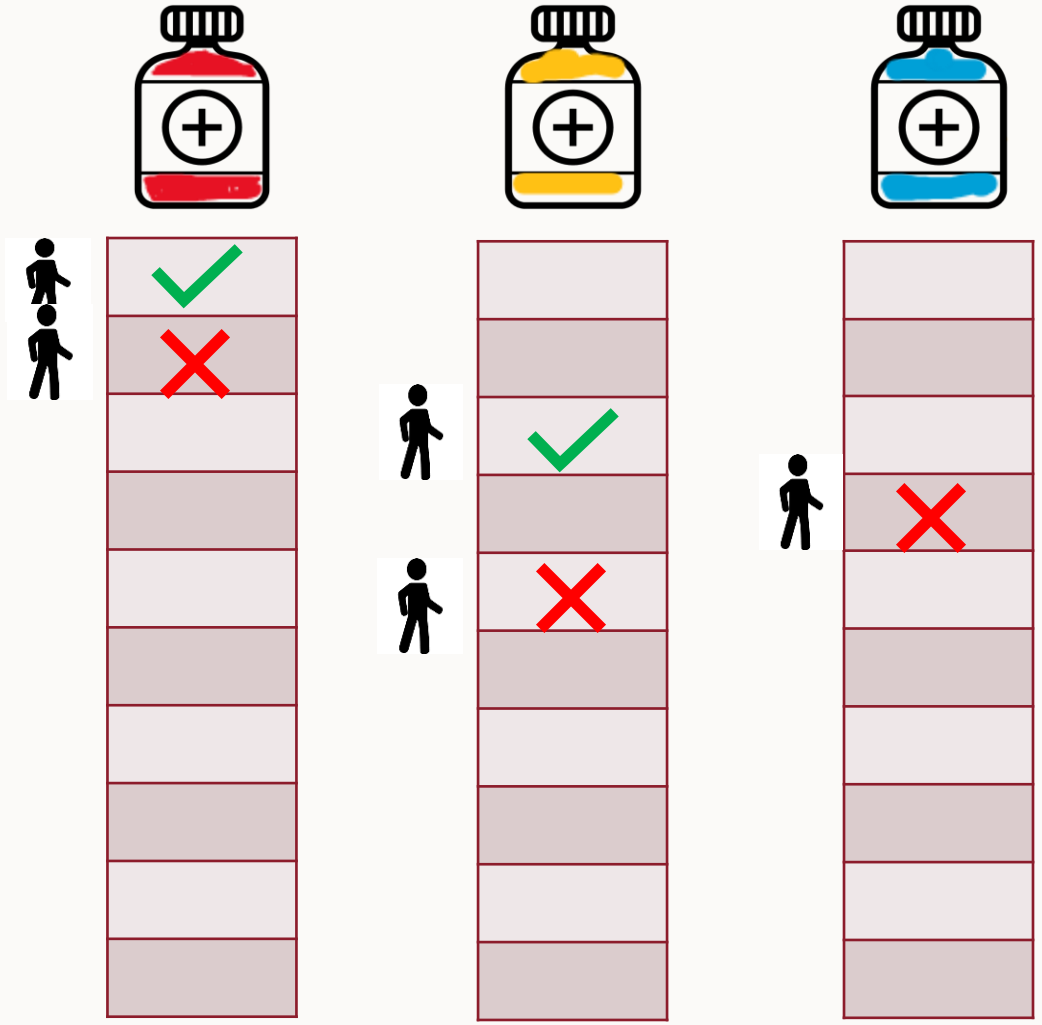
As the denominator → ∞, $Q_t(a) \to q^*(a)$

EXAMPLE · RANDOM SELECTION

Building the estimates

A reward of 1 if treatment succeeds,
0 otherwise

$$Q_t(a) = \frac{\sum_{i=1}^{t-1} R_i}{t-1}$$



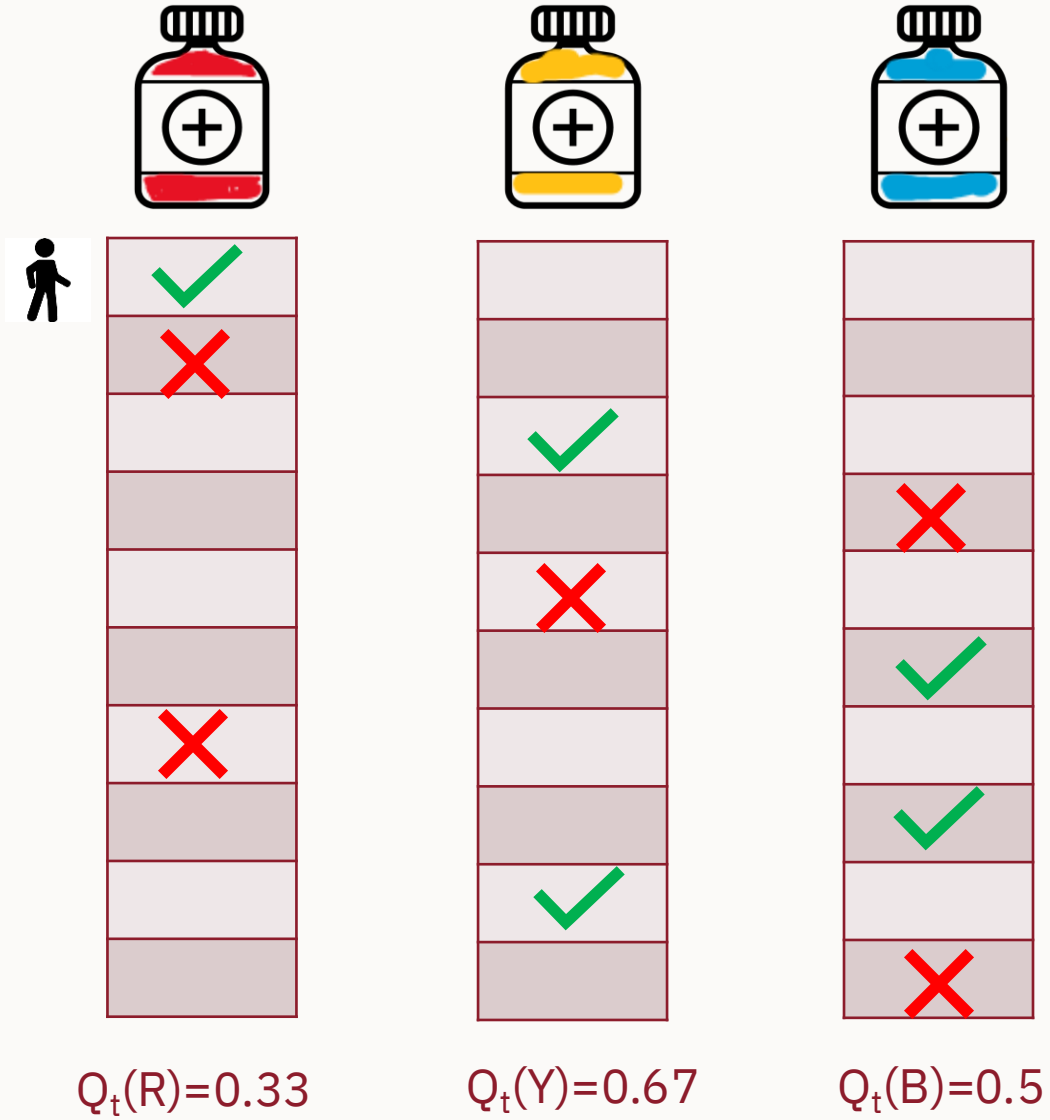
EXAMPLE · RANDOM SELECTION

After ten patients

A reward of 1 if treatment succeeds,
0 otherwise

$$Q_t(a) = \frac{\sum_{i=1}^{t-1} R_i}{t-1}$$

Actions are selected randomly



EXAMPLE · RANDOM SELECTION

Ten patients, treatments chosen at random

Reward = **1** if the treatment succeeds, **0** otherwise. Each row lists the outcomes of the patients who got that treatment.

Red	1 0 0	Q(R) = (1+0+0) / 3 = **0.33**
Yellow	1 0 1	Q(Y) = (1+0+1) / 3 = **0.67**
Blue	0 1 1 0	Q(B) = (0+1+1+0) / 4 = **0.5**

Each estimate uses **only** the patients who received that treatment. With so few samples the estimates are still very noisy.

WHERE WE ARE

Next: how to select an action

How to estimate action values

- ✓ Sample average
- ○ Incremental update
- ○ Nonstationary: constant step size α

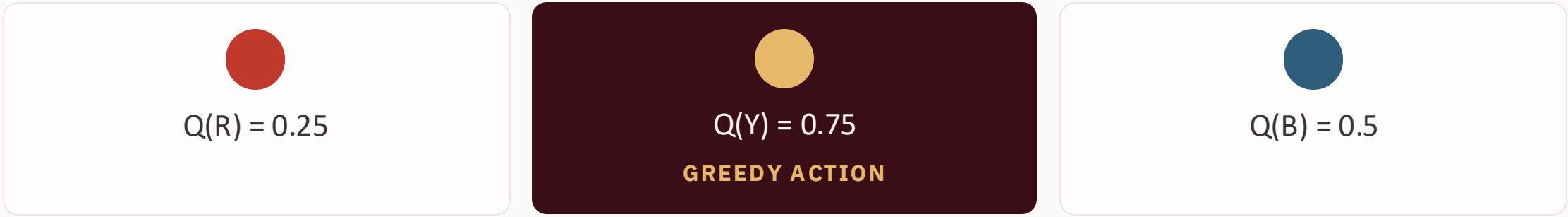
How to select an action

- ✓ Random
- → **Greedy**
- → **ε-greedy**
- ○ Optimistic initial values

ACTION SELECTION

The greedy action

A real doctor would not assign treatments at random: they would prescribe the one they **currently believe** is best. The greedy action is the one with the largest estimated value.



$$A_t \doteq \operatorname{argmax}_a Q_t(a)$$

ties broken arbitrarily, e.g. at random

ACTION SELECTION

Greedy only exploits, and can get stuck

Pure exploitation

Uses current knowledge to maximize the **immediate** reward

No sampling

Never tries apparently inferior actions to check whether they are better

Stuck

Can lock onto a **suboptimal** action forever

WHY IT HAPPENS

True values: $q^*(R) = 0.25$, $q^*(Y) = 0.5$, **$q^*(B) = 0.75$**. Blue's first patient is a failure → $Q(B) = 0$, while Yellow got a lucky start → $Q(Y) = 0.75$. Greedy prescribes Yellow from now on: **Blue is never tried again** and its estimate is never corrected.

ACTION SELECTION

ε-greedy: mostly greedy, sometimes random

at each step t

1 − ε	**Exploit**: $A_t = \text{argmax}_a\ Q_t(a)$
ε	**Explore**: A_t uniformly at random among **all** k actions, ignoring the estimates

Every action keeps being tried

As $t \to \infty$, every $N_t(a) \to \infty$, so every $Q_t(a) \to q^*(a)$

A stochastic policy

Greedy is deterministic, ε-greedy is stochastic (lecture 1). The random choice may also pick the greedy action

ACTION SELECTION

How large should ε be?

ε = 0

Pure greedy

Good at first, then may stay stuck on a suboptimal action

ε = 0.01

Explores little

Slow to find the best action, but exploits it well in the long run

ε = 0.1

Explores more

Finds the best action sooner, but keeps wasting ~10% of the steps

ε = 1

Pure random

Learns all values, never uses them

It depends on the task

Noisier rewards or a longer horizon → exploration pays off more. Next lecture we **measure** it on a testbed

A common trick

Start with a large ε and **decrease it over time**: the "explore early, exploit later" recipe of lecture 1

EXERCISE 1

In ε-greedy action selection, with two actions and $\varepsilon = 0.5$, what is the probability that the greedy action is selected?

Hint: is 0.5 the right answer? Look again at what happens in the random case.

EXERCISE 2 · SUTTON & BARTO 2.2

A k-armed bandit with **k = 4** actions (1, 2, 3, 4) uses **ε-greedy** selection, **sample-average** estimates and initial estimates **$Q_1(a) = 0$** for all a.

At each step t the agent selects one action A_t and receives the reward **R_t of that single action**, not a running total.

On some steps the ε case may have occurred, so the action was selected at random.

On which time steps did this **definitely** occur? On which could it **possibly** have occurred?

STEP	ACTION	REWARD R_t
t=1	$A_1=1$	$R_1=1$
t=2	$A_2=2$	$R_2=1$
t=3	$A_3=2$	$R_3=2$
t=4	$A_4=2$	$R_4=2$
t=5	$A_5=3$	$R_5=0$

WORKED EXAMPLE · SMART HOME

An ε-greedy reader, step by step

3 arms
Tags T1, T2, T3: query one per slot

Reward
+1 new data · −1 nothing new

Selection
ε-greedy, ε = 0.2 · ties broken at random

Estimates
sample average · all Q start at 0

The ε-dice and the random picks are **given**, so everybody gets the same numbers. Q values are those **before** the query.

t	Q(T1)	Q(T2)	Q(T3)	greedy set	ε-dice	queried	R	update of the queried tag
1	0	0	0	{T1,T2,T3} → T1	exploit	T1	+1	**Q(T1) = 1/1 = 1**
2	1	0	0	{T1}	exploit	T1	+1	**Q(T1) = (1+1)/2 = 1**
3	1	0	0	{T1}	**explore → T3**	T3	−1	**Q(T3) = −1/1 = −1**
4	1	0	−1	{T1}	exploit	T1	−1	**Q(T1) = (1+1−1)/3 = 0.33**

…and four more queries

t	Q(T1)	Q(T2)	Q(T3)	greedy set	ε-dice	queried	R	update of the queried tag
5	0.33	0	−1	{T1}	**explore → T3**	T3	+1	**Q(T3) = (−1+1)/2 = 0**
6	0.33	0	0	{T1}	exploit	T1	−1	**Q(T1) = (1+1−1−1)/4 = 0**
7	0	0	0	{T1,T2,T3} → T3	exploit	T3	+1	**Q(T3) = (−1+1+1)/3 = 0.33**
8	0	0	0.33	{T3}	exploit	T3	+1	**Q(T3) = (−1+1+1+1)/4 = 0.5**
end	**0**	**0**	**0.5**	**{T3}**				**T3 is now the greedy tag**

Eight queries: 6 exploit, 2 explore. After step 3, T3 looked like the **worst** tag (Q = −1). The exploration at step 5 brought it back to 0; from step 7 it wins and becomes the greedy choice.

WORKED EXAMPLE · SMART HOME

What the reader learned

	T1	T2	T3
p = P(new data) (hidden)	0.5	0.2	0.8
True value q* = 2p − 1	0	−0.6	**+0.6**
Estimate Q after 8 steps	0	**0 (never queried)**	**0.5**

WHY 2p − 1

R = +1 with probability p
R = −1 with probability 1 − p

$q^* = p\cdot(+1) + (1-p)\cdot(-1) = 2p - 1$

Estimation

Each query updates only the queried tag: the average of its ±1 rewards. T1 already matches q*

A bad first impression

At step 3 the best tag, T3, got −1 and looked the worst. Pure greedy would never query it again

Exploration fixes it

The random query at step 5 gave T3 a second chance: now it is greedy, and 0.5 → 0.6

Still unknown

T2 was never queried: its Q is still the initial 0, not −0.6. ε keeps trying it, now and then

The reader never sees p or q*: it only sees the ±1 rewards, and Q is its guess of q*.

WHERE WE ARE

Next: estimating values efficiently

How to estimate action values

- ✓ Sample average
- → **Incremental update**
- ○ Nonstationary: constant step size α

How to select an action

- ✓ Random
- ✓ Greedy
- ✓ ε-greedy
- ○ Optimistic initial values

INCREMENTAL IMPLEMENTATION

Can we average without storing every reward?

Naive sample average

Store all rewards, re-sum them at every step: memory and computation **grow** with time

Incremental

Keep only the current estimate and a counter: **constant** memory and time per step. Ideal for a sensor node

SIMPLIFIED NOTATION: FOCUS ON A SINGLE ACTION

$\mathbf{R_i}$ = reward received after the i-th selection of *this* action

$\mathbf{Q_n}$ = estimate after it has been selected n − 1 times: $Q_n = (R_1 + R_2 + \dots + R_{n-1}) / (n - 1)$

INCREMENTAL IMPLEMENTATION

Deriving the update rule, step by step

Recall: $\mathbf{Q_n = (1/(n-1)) \sum_{i=1}^{n-1} R_i}$ is the average of the first n−1 rewards, so $\mathbf{\sum_{i=1}^{n-1} R_i = (n-1)\, Q_n}$

#	Formula	What we do
0	$Q_{n+1} = (1/n) \sum_{i=1}^{n} R_i$	Start from the definition: average of the first n rewards
1	$= (1/n)\,(R_n + \sum_{i=1}^{n-1} R_i)$	Take the last reward R_n out of the sum
2	$= (1/n)\,(R_n + (n-1) \cdot (1/(n-1)) \sum_{i=1}^{n-1} R_i)$	Multiply and divide the sum by (n−1)
3	$= (1/n)\,(R_n + (n-1)\, Q_n)$	Substitute: $(1/(n-1)) \sum_{i=1}^{n-1} R_i$ is exactly Q_n
4	$= (1/n)\,(R_n + n\, Q_n - Q_n)$	Expand $(n-1)\, Q_n = n\, Q_n - Q_n$
5	$= (1/n) \cdot n\, Q_n + (1/n)\,(R_n - Q_n)$	Split the term $n\, Q_n$ from the rest
6	$= Q_n + (1/n)\,[R_n - Q_n]$	Simplify n/n = 1: the incremental update

Holds even for n = 1: $Q_2 = Q_1 + 1 \cdot [R_1 - Q_1] = R_1$, whatever Q_1 was. Per step we store only two numbers, Q_n and n.

INCREMENTAL IMPLEMENTATION

The general form of the update rule

GENERAL RULE

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize} \cdot [\text{Target} - \text{OldEstimate}]$$

$$Q_{n+1} = Q_n + 1/n \cdot [R_n - Q_n]$$

Target = R_n

Where we want to move: the new reward (a noisy one)

$[R_n - Q_n]$ = error

How far the old estimate Q_n is from the target: we take a step to reduce it

StepSize = 1/n

How far to move. For the sample average it is 1/n and shrinks every step. In general: α

Intuition: reward higher than expected → raise the estimate; lower → decrease it. A large error moves the estimate more.

PUTTING IT TOGETHER

A simple bandit algorithm

Initialize, for a = 1 to k:

$Q(a) \leftarrow 0$

$N(a) \leftarrow 0$

Loop forever:

$A \leftarrow \text{argmax}_a\, Q(a)$ w.p. $1 - \varepsilon$

random action w.p. ε

$R \leftarrow \text{bandit}(A)$

$N(A) \leftarrow N(A) + 1$

$Q(A) \leftarrow Q(A) + (1/N(A)) \cdot [R - Q(A)]$

Select

ε-greedy, ties broken randomly

Act

bandit(a) takes the action and returns its reward

Learn

incremental sample average: only the chosen action is updated

NONSTATIONARY PROBLEMS

When the rewards change over time

Stationary

Reward distributions fixed over time: averaging over all the past is fine

Nonstationary

Distributions change: old rewards describe a world that no longer exists

DOCTOR TRIALS

Blue is more effective in the **winter** months



The doctor does not know about the change but wants to **adapt**. With 1/n, after hundreds of patients a winter reward barely moves Q(B): the estimate reacts too slowly.

NONSTATIONARY PROBLEMS

Tracking: use a constant step size

$$Q_{n+1} \doteq Q_n + \alpha \left[R_n - Q_n \right]$$

$\alpha \in (0, 1]$ constant

1/n vs α

1/n shrinks to 0 and the estimate freezes; a constant α keeps it **reactive** forever

Recent rewards count more

The most recent rewards weigh more than older ones

Trade-off

Large α: adapts fast but noisy. Small α: smooth but slow. $\alpha = 1$: $Q_{n+1} = R_n$

NONSTATIONARY PROBLEMS

An exponential recency-weighted average

$$Q_{n+1} = Q_n + \alpha[R_n - Q_n]$$

$$= \alpha R_n + (1-\alpha)Q_n$$

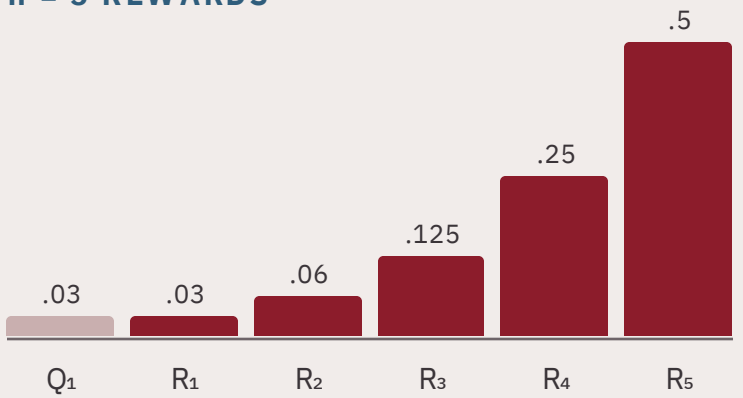
$$= \alpha R_n + (1-\alpha)[\alpha R_{n-1} + (1-\alpha)Q_{n-1}]$$

$$= \alpha R_n + (1-\alpha)\alpha R_{n-1} + (1-\alpha)^2 Q_{n-1}$$

$$= \ldots$$

$$= (1-\alpha)^n Q_1 + \Sigma_{i=1}^{n} \alpha(1-\alpha)^{n-i} R_i$$

WEIGHTS WITH STEP SIZE α = 0.5, AFTER n = 5 REWARDS



A weighted average

$(1-\alpha)^n + \Sigma\, \alpha(1-\alpha)^{n-i} = 1$: the weights sum to one

Exponential decay

Each step back multiplies the weight by $(1-\alpha)$; Q_1 fades but is never fully forgotten

EXERCISE 3

Tracking a change: 1/n vs. constant α

Blue becomes more effective in winter. After 20 patients the doctor has **Q(B) = 0.5**. Then the next **4 patients are all cured** (R = 1).

Compute Q(B) after each of the 4 rewards:

(a) sample average, step size 1/n

(b) constant step size α = 0.2

HINT

Q ← Q + step · [R − Q]

With 1/n the next reward is the 21st, so the first step size is 1/21.

SOLUTION · EXERCISE 3

A constant α follows the change

reward	step 1/n	Q with 1/n	Q with α = 0.2
21st	1/21	0.5 + (1/21)(1 − 0.5) = 0.52	**0.5 + 0.2(1 − 0.5) = 0.60**
22nd	1/22	0.55	**0.68**
23rd	1/23	0.57	**0.74**
24th	1/24	0.58	**0.80**

1/n: the past outweighs the present

The 20 old rewards dominate: to reach 0.8 it would take 30 cures in a row

Constant α: the past fades

Each old reward is multiplied by 0.8 at every step: 0.8 after just 4 cures

CONCEPT CHECK

True or false?

T / F ?	**1.** Greedy selection never explores.
T / F ?	**2.** With $\varepsilon = 0.1$ the greedy action is chosen 90% of the time.
T / F ?	**3.** The goal of a bandit is to discover the best arm.
T / F ?	**4.** With the sample average, Q_1 influences the estimate forever.

CONCEPT CHECK

True or false: answers

TRUE

Greedy selection never explores.
It always takes the argmax: only ties are broken at random.

FALSE

With ε = 0.1 the greedy action is chosen 90% of the time.
$1 - \varepsilon + \varepsilon/k$: the random pick can land on it too (91% with k = 10).

FALSE

The goal of a bandit is to discover the best arm.
The goal is the total reward collected while learning: exploring has a cost.

FALSE

With the sample average, Q_1 influences the estimate forever.
$Q_2 = Q_1 + 1 \cdot [R_1 - Q_1] = R_1$: Q_1 is forgotten after the first try (not with a constant α).

WHERE WE ARE

Last: optimistic initial values

How to estimate action values

- ✓ Sample average
- ✓ Incremental update
- ✓ Nonstationary: constant step size α

How to select an action

- ✓ Random
- ✓ Greedy
- ✓ ε-greedy
- → **Optimistic initial values**

OPTIMISTIC INITIAL VALUES

An optimistic doctor

All methods so far depend on the initial estimates $Q_1(a)$. We can use them as a simple way to **encourage exploration**.

BEFORE

$Q_1(a) = 0$

Not optimistic: any success already looks "good"

NOW

$Q_1(a) = 2$

Above any possible reward (max 1): every treatment assumed excellent until shown otherwise

1 Prescribe treatments greedily: all look equally great → 2 Any real outcome lowers the estimate of the one tried → 3 The untried ones now look better: switch to them

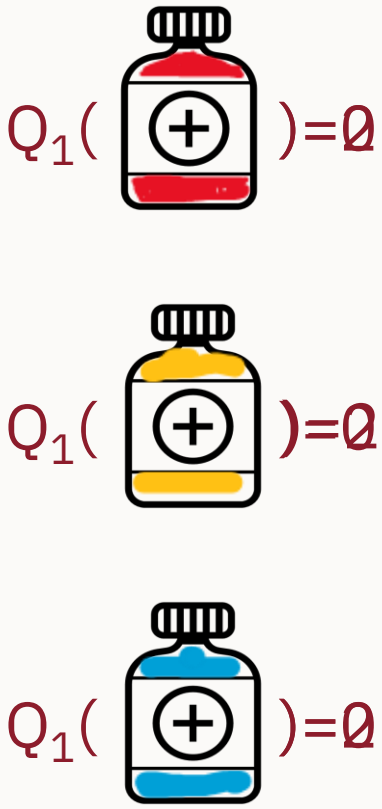
EXAMPLE · OPTIMISM

Start optimistic

Previously the initial estimated values were assumed to be 0, which is not necessarily optimistic

Now, our doctor optimistically assumes that each treatment is highly effective before running the trial.

let's make the initial value for each action 2

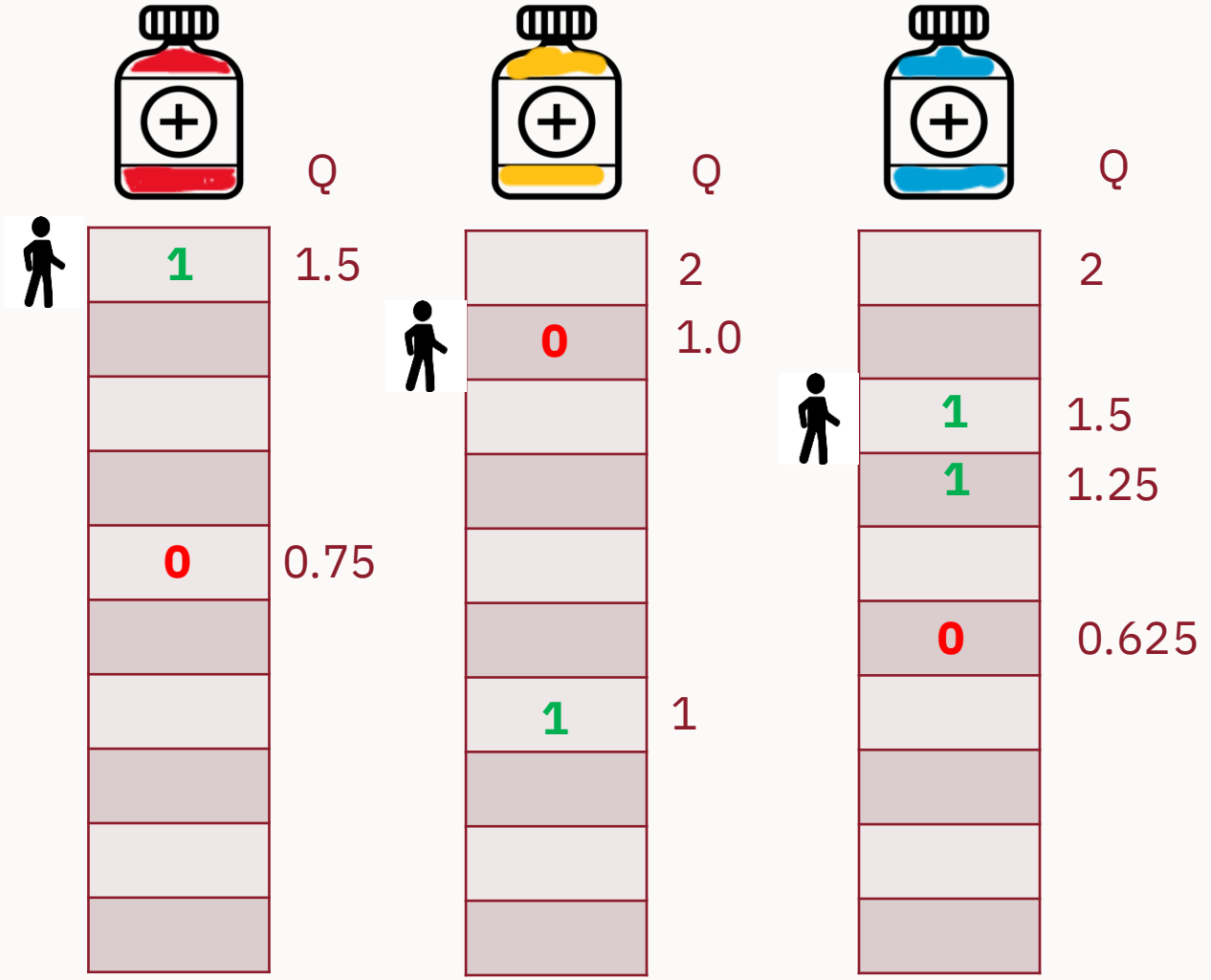


Updates with α = 0.5

A reward of 1 if treatment succeeds, 0 otherwise

$$Q_{n+1} \doteq Q_n + \alpha\left[R_n - Q_n\right]$$

Let α=0.5
$Q_1(a)=2$



EXAMPLE · GREEDY WITH $Q_1(a) = 2$, STEP SIZE $\alpha = 0.5$

Optimism drives exploration

Reward 1 if the treatment succeeds, 0 otherwise. Update: $Q \leftarrow Q + 0.5 \cdot [R - Q]$. Values **after** each step; the updated one is highlighted.

t	greedy choice	R	Q(Red)	Q(Yellow)	Q(Blue)
0	—	—	2	2	2
1	Red (tie)	1	1.5	2	2
2	Yellow (tie)	0	1.5	1.0	2
3	Blue	1	1.5	1.0	1.5
4	Blue (tie)	1	1.5	1.0	1.25
5	Red	0	0.75	1.0	1.25
6	Blue	0	0.75	1.0	0.625
7	Yellow	1	0.75	1.0	0.625

Purely greedy, yet all three treatments were tried in the first 3 steps, and the leader keeps changing.

OPTIMISTIC INITIAL VALUES

A learner that is always disappointed

Disappointment

Whatever action is chosen, the reward is below the estimate: the learner switches to the others

Everything is tried

All actions are tried several times before the estimates converge

Exploration for free

A fair amount of exploration even if greedy actions are selected all the time

LIMITATIONS

- Exploration happens **only at the beginning**: useless if the problem is nonstationary and changes later
- We must know roughly how large the rewards can be, to choose a Q_1 that is truly optimistic
- With sample averages Q_1 is forgotten after the first try: it works best with a constant α

PUTTING IT TOGETHER

The same algorithm, three knobs

Initialize, for a = 1 to k:

Q(a) ← Q_1

N(a) ← 0

Loop forever:

A ← argmax_a Q(a) w.p. 1 − ε

random action w.p. ε

R ← bandit(A)

N(A) ← N(A) + 1

Q(A) ← Q(A) + α · [R − Q(A)]

Q_1 · initial value

0 = neutral; high = optimistic, explores early

ε · exploration

0 = greedy; small = keeps exploring forever

α · step size

1/N(A) = sample average (stationary); constant = tracks changes

BACK TO NETWORKS

Bandits in autonomous networks

Smart-home reader

Arms: the battery-free tags (lecture 1)
Reward: +1 new data, −1 nothing new

Channel selection

Arms: radio channels
Reward: 1 if the packet is delivered, 0 if it collides

MAC parameters

Arms: duty cycles, contention windows, Tx power
Reward: throughput per joule

Why bandits fit

Unknown and noisy rewards, learned online; tiny memory and computation, fine for battery-free and sensor nodes

What is missing

A tag's chance of having new data grows with the time since its last query: the reward depends on a **state** → Q-learning

CONCLUSIONS

How can we balance exploration with exploitation?

ANSWER 1 · ε-GREEDY

Explore **a little, forever**: a random action with probability ε, the greedy one otherwise

ANSWER 2 · OPTIMISM

Explore **a lot, at the start**: high Q_1, then pure greedy; disappointment does the rest

Both need good estimates

$Q \leftarrow Q + \alpha[R - Q]$: $\alpha = 1/n$ if the world is stationary, constant α if it changes

In the network

A reader, a channel, a MAC parameter: few numbers per arm, learned online

Next: **which answer works better?** Comparing them on the 10-armed testbed, and a smarter way to explore: **UCB**, optimism in the face of uncertainty.